1) O referinta nu trebuie initializata atunci cand este definita in urmatoarele cazuri:

A) Daca nu este membra a unei clase;
B) Daca nu este o valoare returnata;
C) Daca nu este parametru de functie;
D) Toate raspunsurile nu sunt corecte;

2) Se poate obtine adresa unei referinte(A/F)

3) Care dintre urmatoarele linii vor da eroare de eroare de compilare:

```
1  #include <iostream>
2
3
4  int sum(int &a , int * b )
5  {
6      return a + *b;
7
8  }
9
10 int main()
11 {
12     int x = 9 , y = 12 ;
13   int a = sum (&y,x);
14   int b = sum(&x,*y);
15   int c = sum(5,5);
16   return 0 ;
17 }
18
19
```

A) 13,14
B) 13,14,15
C) 14,15
D) Programul va compila, dar va avea runtime error

4) Supraincarcarea functiilor este un caz de polimorfism la executie(A/F).
5) In cazul supraincarcarii functiilor, tipul de intoarcere al unei functii nu este suficient intotdeauna pentru a face diferenta(A/F).

6) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
1   #include <iostream>
2
3   class Cls
4   {
5   int a,x,z;
6   public  :
7       Cls(int a,int z):
8           a(a),z(z){std::cout << 2 << "\n";}
9       Cls(int a,int z ,int x = 2026):
10          a(a),z(z),x(x){std::cout << 2026 << "\n";}
11      Cls(int a,int z,double x):
12          a(a),z(z),x(x){std::cout << 3 << "\n";}
13      int sum()
14      {
15          return a + x + z ;
16      }
17  }o(2,3);
18
19  int main()
20  {
21    Cls o(1,1,1);
22    std::cout << o.sum() << "\n";
23    return 0 ;
24  }
```

Nu compileaza, linia 17 - Eroare de ambiguitate la crearea unui obiect cu 2 parametrii, din cauza constructorului de la linia 7 si 9. Stergerea valorii implicite sau stergerea o(2,3).

0.3p - eroare ambiguitate, 0.2p -modificare

7) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
1  #include <iostream>
2
3  class Cls
4  {
5  int a,x,z;
6  public  :
7      Cls(int a,int z):
8          a(a),z(z){std::cout << 2 << "\n";}
9      Cls(int a,int z ,int x = 2026):
10         a(a),z(z),x(x){std::cout << 2026 << "\n";}
11     Cls(int a,int z,double x):
12         a(a),z(z),x(x){std::cout << 3 << "\n";}
13     int sum()
14     {
15         return a + x + z ;
16     }
17 }o1(1,1,static_cast<double>(1)),o2(1,1,1),o3(1,1,static_cast<int>(1.1)),o4,o5(2,2,2.5);
18
19 int main()
20 {
21   Cls * o = new Cls[5]{o1,o2,o3,o4,o5};
22
23   return 0 ;
24 }
25
```

Nu compileaza, linia 17 - Eroare la crearea obiectului o4 - nu exista constructor fara parametri. Ii adaugam lui o4 3 parametrii/ Scriem constructor fara parametri.

0.3p - eroare ambiguitate, 0.2p -modificare, 0.1 - Daca se adauga doar 2 parametrii la o4

8) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
1  #include <iostream>
2
3  class Cls
4  {
5  int a,x,z;
6  public  :
7      Cls(int a,int z):
8          a(a),z(z){std::cout << 2 << "\n";}
9      Cls(int a = 2026,int z ,int x):
10         a(a),z(z),x(x){std::cout << 2026 << "\n";}
11     Cls(int a,int z,double x):
12         a(a),z(z),x(x){std::cout << 3 << "\n";}
13     int sum()
14     {
15         return a + x + z ;
16     }
17 }o1(1,1,static_cast<double>(1)),o2(1,1,1),o3(1,1,static_cast<int>(1.1)),o5(2,2,2.5);
18
19 int main()
20 {
21   Cls * o = new Cls[4]{o1,o2,o3,o5};
22   delete[] o ;
23
24   return 0 ;
25 }
```

Nu compileaza, linia 9 - Eroare, valorile implicite se completeaza de la dreapta la stanga. 0.2p - Inlocuim a = 2026 cu a , 0.1p - sau dam valori default si pt z si x.

9) Diferenta dintre class si struct este:

A) Class are datele membre private implicit, iar struct protected implicit;
B) Class are datele membre publice implicit, iar struct private implicit;
C) Class are datele membre private implicit, iar struct publice implicit;
D) Class are datele membre protected implicit, iar struct publice implicit;

10) Struct nu poate participa la mostenire(A/F).
11) Struct nu poate avea date membre protected(A/F).
12) Struct nu poate avea functii membre, ci doar date membre(A/F).
13) Struct nu poate avea functii membre virtuale(A/F).
14) Struct nu poate participa la mostenire polimorfica(A/F).

15) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```cpp
#include <iostream>

struct Baza
{
    int a,b;
     void afis()
    {
        std::cout << a << " " << b << "\n";
    }
    Baza(int a, int b):a(a),b(b){}
};

struct Deriv1 : Baza
{
    int c,d;
    void afis ()
    {
        Baza::afis();
        std::cout << c << " " << d << "\n";
    }
    Deriv1(int a, int b, int c, int d ): Baza(a,b),c(c),d(d){}

};

int main()
{
    Deriv1 deriv1(1,1,1,1);
    deriv1.afis();
    Baza * baza = new Deriv1(1,deriv1.b,3,4);
    baza->afis();
    return 0 ;
}
```

Programul compileaza si afiseaza :
1 1
1 1
1 1
0.5 p - Raspunsul corect ,

0.1p -

1 1
1 1
1 1
3 4

0p - orice altceva

16) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
1   #include <iostream>
2
3   class A
4   {
5       public :
6       A(){std::cout << "A ";}
7       ~A(){std::cout << "~A ";}
8   };
9
10  struct B: A
11  {
12      public :
13      B(){std::cout << "B ";}
14     virtual ~B(){std::cout << "~B ";}
15  };
16
17  class C : public B
18  {public :
19       C(){std::cout << "C ";}
20      ~C(){std::cout << "~C ";}
21  }o;
22
23  int main()
24  {
25      C *c = new C;
26      A *a = new C; delete a;
27      return 0;
28  }
```

Programul compileaza si va afisa
0.5p - A B C A B C A B C ~A ~C ~B ~A
0.1p - A B C A B C A B C ~C ~B ~A ~C ~B ~A

17) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
#include <iostream>

class A
{
    public :A(){std::cout << "A ";}
    ~A(){std::cout << "~A ";}
};

struct B: virtual A
{    public :B(){std::cout << "B ";}
   virtual ~B(){std::cout << "~B ";}
};

class C : virtual public A
{public :C(){std::cout << "C ";}
    ~C(){std::cout << "~C ";}}o;

struct D : B,C
{D(){std::cout << "D ";}
    ~D(){std::cout << "~D ";}};
int f(A a)
{return 1;}
int f(A* a)
{return 2;}

int main()
{
    D * d = new D(); delete d;
    A * a = new A(); f(*a);f(a);return 0;
}
```

Programul compileaza si va afisa : 0.5p - A C A B C D ~D ~C ~B ~A A ~A ~C ~A

0.1p - A C A B C D ~D ~C ~B ~A A ~A A ~A ~C ~A

18) Intr-o functie membra statica se pot folosi date membre nestatice(A/F)
19) Intr-o functie membra nestatica poate sa apeleze o functie membra statica(A/F)
20) O constanta statica trebuie sa fie declarata in clasa si definita in afara acesteia(A/F)
21) Cate dintre afirmatiile de mai jos sunt adevarate:
I) O functie statica poate avea ca parametru obiecte de la care sa foloseasca o data membra statica
II) O functie statica poate avea un obiect local de la care sa foloseasca o data membra statica.
III) O functie statica nu poate sa fie virtuala

A)0
B) 1
C) 2
D) 3

19) Clasele locale din functii pot accesa varibilele locale ale acestora(A/F)
20) Clasele locale din functii pot accesa variabilele statce locale ale acestora(A/F)
21) Clasele locale din functii nu pot avea functii membre automat inline(A/F)
22) Clasele “nested”(clasa in clasa) pot avea date membre statice(A/F)
23) Clasele “nested”(clasa in clasa) nu pot avea functii membre statice(A/F)
24) Operator de rezolutie de scop si sizeof pot fi incarcati doar ca functii membre (A/F)
25) Prin supraincarcare se poate schimba atat precedenta operatorilor cat si numar de operanzi ai acestor(A/F)
26) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```
1  #include <iostream>
2
3  class Baza
4  {
5      protected:
6      int a , b ;
7  public :
8      Baza(int a, int b):a(a),b(b){}
9        static int suma()
10     {
11         return this->a + this->b;
12     }
13 };
14
15
16 class Deriv : public Baza
17 {
18 public :
19     Deriv(int a,int b): Baza(a,b){}
20     int operator+(int c)
21     {
22         return this-> a + c;
23     }
24 };
25
26 int main()
27 {
28     Deriv x(1,2); Baza y(3,4);
29     std::cout << x + y.suma();
30 }
```

0.3p Eroare la linia - 11, functia statica nu are acces la *this. 0.2p - eliminare static

27) Daca compileaza ce va afisa, in caz contrar propuneti o singura modificare prin care programul devine corect.

```cpp
#include <iostream>

template <class T>
// A - ASCI = 65
void f(T a, T b)
{
    std::cout << a*a+2*a*b+b*b << "\n";
}
template <> void f(double a, double b)
{
    std::cout << a+b+0.111112 << "\n";
}

void f(int a, int b)
{
    std::cout << a+b+3.14 << "\n";
}

int main()
{
    f(1.2,3.4);
    f(5,6);
    f('A',7.5);
    return 0 ;
}
```

Programul va compila si va afisa:
4.71111
14.14
75.14

28) Ce se întâmplă dacă o excepție este aruncată într-o funcție și **nu este prinsă** în acea funcție?

A) Programul continua executia normal
B) Excepția se propagă către funcția apelata
C) Excepția se propagă către funcția apelantă
D) Programul intră într-o buclă infinită

29) În C++, o excepție poate fi aruncată sub forma unui **obiect al unei clase definite de utilizator**.

A / F

30) În C++, este recomandat să se arunce excepții sub forma **pointerilor către obiecte**, pentru a evita copierea acestora.

A / F